Batch Loss Score for Dynamic Data Pruning

Qing Zhou[1], Bingxuan Zhao[1], Tao Yang[1], Hongyuan Zhang[2], Junyu Gao[1], Qi Wang[1]*
[1]Northwestern Polytechnical University
[2]The University of Hong Kong
{chautsing, bxuanzhao202, ytao9464, hyzhang98, gjy3035, crabwq}@gmail.com

Abstract

*Dynamic data pruning accelerates deep learning by selectively omitting less informative samples during training. While per-sample loss is a common importance metric, obtaining it can be challenging or infeasible for complex models or loss functions, often requiring significant implementation effort. This work proposes the Batch Loss Score (BLS), a computationally efficient alternative using an Exponential Moving Average (EMA) of readily available batch losses to assign scores to individual samples. We frame the batch loss, from the perspective of a single sample, as a noisy measurement of its scaled individual loss, with noise originating from stochastic batch composition. It is formally shown that the EMA mechanism functions as a first-order low-pass filter, attenuating high-frequency batch composition noise. This yields a score approximating the smoothed and persistent contribution of the individual sample to the loss, providing a theoretical grounding for BLS as a proxy for sample importance. BLS demonstrates remarkable code integration simplicity (**three-line injection**) and readily adapts existing per-sample loss-based methods (**one-line proxy**). Its effectiveness is demonstrated by enhancing two such methods to losslessly prune **20%-50%** of samples across 14 datasets, 11 tasks and 18 models, highlighting its utility and broad applicability, especially for complex scenarios where per-sample loss is difficult to access. Code is available at* `https://github.com/mrazhou/BLS`.

1. Introduction

The training of state-of-the-art deep neural networks often involves massive datasets and extensive computation, posing significant resource challenges [6, 10, 20, 37, 40, 53]. Dynamic data pruning techniques [36, 54] aim to mitigate this burden by improving training efficiency. These methods [8, 17, 31, 38] dynamically identify and selectively skip samples presumed less informative at different training stages, thereby focusing computational resources.

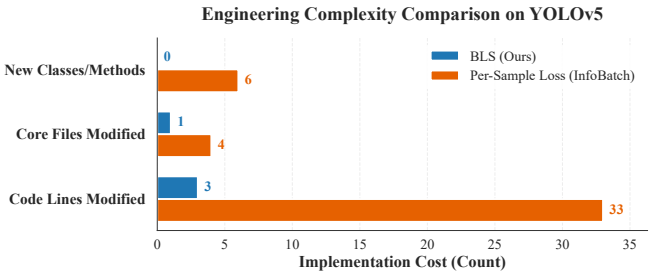

Figure 1. Engineering complexity comparison on YOLOv5. BLS: 3-line plug-in | InfoBatch: 33+ lines + intrusive modifications.

A dominant strategy leverages per-sample loss $l_i(t)$ to gauge sample importance [17, 30, 34, 36, 38, 54], driven by the intuition linking higher loss to more informative samples. **However, directly accessing $l_i(t)$ presents significant practical barriers**. Standard deep learning pipelines are highly optimized around computing the *mean batch loss*, $L(\mathcal{B}_t, t)$, for gradient updates, making the recovery of individual $l_i(t)$ values post-aggregation non-trivial. This challenge is exacerbated by modern frameworks [33] that abstract loss calculations and by complex objectives (e.g., multi-component losses in object detection [39]) where defining and disentangling a representative per-sample scalar from the aggregate requires deep, task-specific knowledge and modifications [4, 5, 29, 39, 46]. Thus, while $L(\mathcal{B}_t, t)$ is ubiquitously available, the per-sample $l_i(t)$ remains largely inaccessible in many practical scenarios. This fundamental inaccessibility—spanning both engineering friction and conceptual difficulties—forms a critical bottleneck, significantly hindering the development and deployment of powerful loss-aware dynamic training strategies. Consequently, developing reliable methods to infer sample importance *without* direct per-sample loss dependence is vital for advancing more adaptable and resource-efficient deep learning.

To circumvent these challenges, we introduce the Batch Loss Score (BLS), a novel mechanism ***inferring sample importance solely from the readily available mean batch loss***. BLS leverages the unique temporal sequence of batch losses

*Corresponding author.

encountered by each sample. It maintains per-sample scores, updated via an Exponential Moving Average (EMA) only when a sample is present in the current batch, incorporating that batch's mean loss. This sample-specific, conditional EMA update allows each score to reflect a unique participation history. As we demonstrate theoretically, this process inherently filters high-frequency noise stemming primarily from stochastic batch composition. The resulting smoothed score thus serves as a robust proxy for a sample's persistent loss contribution, achieving this with negligible computational overhead and exceptional implementation simplicity *(requiring only 3 lines of code, detailed in supplementary material Secs. 7 and 8)*, facilitating easy integration into existing workflows. As shown in Figure 1, BLS is significantly more simpler than existing methods, requiring only 3 lines of code to implement, while InfoBatch requires 33+ lines of code and intrusive modifications to the training pipeline.

The contributions of this paper are threefold. **First**, we introduce BLS, a practical and efficient method for scoring sample importance using only mean batch losses. **Second**, we provide a rigorous theoretical justification, analyzing BLS through a signal-processing lens and proving its noise-filtering properties under the frequency separation hypothesis. **Third**, we conduct extensive empirical validation across diverse tasks and architectures, showing that BLS achieves losslessly training speedups when integrated into existing per-sample loss-based pruning methods, confirming its efficacy as a proxy, robust generalization, and utility without requiring per-sample loss access.

2. Related Work

Dynamic Data Selection. The idea of focusing computation on informative samples has a rich history, dating back to curriculum learning concepts [3], where models are initially trained on easier samples. Modern dynamic data selection methods automate this process throughout training. Many approaches [23, 31, 45] aim to identify and prioritize hard or informative samples. For instance, InfoBatch [36] implements soft pruning of samples exhibiting lower-than-average loss and corrects for the induced bias by rescaling gradients. SeTa [54] utilizes curriculum learning to select high-value samples in an easy-to-hard progression, achieving lossless acceleration during large-scale data training. These frameworks often rely on specific importance metrics, such as per-sample loss, to guide the selection. BLS contributes to this area by proposing a novel scoring mechanism that can be readily integrated into existing dynamic selection frameworks that rely on sample scores.

Sample Importance Metrics. Various metrics [8, 21, 34, 41] have been proposed to quantify sample importance for data selection. For dynamic data selection, per-sample loss is arguably the intuitive and frequently used metric [30, 36, 45, 54], based on the premise that higher loss indicates greater difficulty or informativeness. Gradient norms [23] or related quantities like gradient variance [2] provide another perspective, measuring the potential impact of a sample on parameter updates. A primary challenge common to many of these metrics, particularly per-sample loss, is implementation complexity associated with obtaining them for every sample at frequent intervals during training. BLS differentiates itself by estimating sample importance indirectly through the temporal dynamics of the easily accessible batch loss, offering a significant advantage in terms of computational and implementation overhead.

Leveraging Batch Statistics. While most selection methods focus on per-sample properties, some research utilizes batch-level statistics for other purposes, such as adaptive optimization (e.g., adaptive batch size methods [42]). However, using temporal statistics of batch loss specifically to derive individual sample scores for data selection appears less explored. BLS is novel in its specific use of EMA-smoothed batch loss history as a direct proxy for individual sample importance over time. It exploits the readily available batch loss signal in a new way, leveraging its temporal dynamics through filtering to overcome the lack of immediate per-sample information.

3. Methodology: Batch Loss Score (BLS)

3.1. Preliminaries and Problem Setup

Let $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ be the training dataset of size N, $f(\cdot;\boldsymbol{\theta})$ denote the model parameterized by $\boldsymbol{\theta} \in \mathbb{R}^d$, and $l(f(\mathbf{x};\boldsymbol{\theta}), \mathbf{y})$ be the per-sample loss function. At training iteration t, the model parameters are $\boldsymbol{\theta}_t$. The instantaneous loss for sample i is $l_i(t) \coloneqq l(f(\mathbf{x}_i;\boldsymbol{\theta}_t), \mathbf{y}_i)$.

Training typically involves sampling mini-batches $\mathcal{B}_t \subset \mathcal{D}$ of size $B = |\mathcal{B}_t|$ at each iteration t. The mean batch loss, used for gradient computation, is:

$$L(\mathcal{B}_t, t) = \frac{1}{B} \sum_{j \in \mathcal{B}_t} l_j(t) \tag{1}$$

Model parameters $\boldsymbol{\theta}_t$ are updated based on the gradient of $L(\mathcal{B}_t, t)$ or a related objective, e.g., $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L(\mathcal{B}_t, t)|_{\boldsymbol{\theta}=\boldsymbol{\theta}_t}$.

Definition 3.1 (Dynamic Data Pruning)**.** Dynamic data pruning is a process where, at certain steps t, an importance score $s_i(t)$ is assigned to each sample $i \in \mathcal{D}$. Based on these scores, a subset $\mathcal{D}'_t \subseteq \mathcal{D}$ is selected for next training cycle (e.g., by pruning samples with $s_i(t)$ below a threshold [36], or selecting samples within a specific difficulty window [54]). Samples in $\mathcal{D} \setminus \mathcal{D}'_t$ are temporarily pruned for the next cycle.

Our objective is to justify using a score derived solely from the sequence of observed $L(\mathcal{B}_t, t)$ values as a meaningful proxy for the unobserved (or expensive) $l_i(t)$ for data pruning decisions.

3.2. Per-Sample Score Estimation

To overcome the lack of sample differentiation in the raw $L(\mathcal{B}_t, t)$ and capture individual sample characteristics over time, we introduce the Batch Loss Score (BLS). Each sample $i \in \mathcal{D}$ is associated with a score $s_i(t)$, initialized (e.g., $s_i(0) = L(\mathcal{B}_0, 0)$). At each training step t, after computing $L(\mathcal{B}_t, t)$, the scores are updated based on batch participation:

Definition 3.2 (Batch Loss Score)**.** The BLS score $s_i(t)$ for sample i at step t is updated via EMA:

$$s_i(t) = \begin{cases} \alpha s_i(t-1) + (1-\alpha) L(\mathcal{B}_t, t) & \text{if } i \in \mathcal{B}_t \\ s_i(t-1) & \text{if } i \notin \mathcal{B}_t \end{cases} \tag{2}$$

where $\alpha \in (0, 1)$ is the EMA decay factor, controlling the memory of the score.

The crucial aspect of Equation (2) is that $s_i(t)$ only incorporates information from batches $\mathcal{B}_\tau$ where $i \in \mathcal{B}_\tau$. This allows $s_i(t)$ to accumulate a unique temporal history for sample i, distinct from other samples $j \neq i$.

3.3. Seamless Proxy Integration

BLS functions as a transparent proxy for existing dynamic pruning frameworks (e.g., InfoBatch, SeTa). Instead of intrusive per-sample loss extraction, BLS calculates $s_i(t)$ via the mean batch loss and feeds it directly into the framework's sampler as a drop-in replacement for the exact loss $l_i(t)$. Consequently, the downstream pruning algorithm remains completely agnostic to the score's origin, requiring zero modifications to its core scheduling logic or hyperparameters.

4. Theoretical Analysis: BLS as Noise-Filtered Smoothed Loss

We first decomposes the mean batch loss $L(\mathcal{B}_t, t)$ into a sample's scaled loss (signal) and batch composition noise (Sec. 4.1). Then show BLS's Exponential Moving Average acts as a low-pass filter, attenuating this noise to estimate the sample's persistent loss contribution (Sec. 4.2).

4.1. Decomposition of Batch Loss

Consider a specific sample i and a batch $\mathcal{B}_t$ containing it ($i \in \mathcal{B}_t$). We can decompose the mean batch loss to isolate the direct contribution of $l_i(t)$:

$$L(\mathcal{B}_t, t) = \underbrace{\frac{1}{B} l_i(t)}_{\text{Scaled Signal } (\mathcal{S}_i)} + \underbrace{\frac{1}{B} \sum_{j \in \mathcal{B}_t, j \neq i} l_j(t)}_{\text{Batch Composition Noise } (\mathcal{N}_i)} \tag{3}$$

This decomposition is central to our analysis. It separates the component directly related to sample i's loss (the *signal*, albeit scaled) from the variability induced by the other samples randomly co-selected in the batch (the *noise*).

Definition 4.1 (Batch Composition Noise)**.** For $i \in \mathcal{B}_t$, the batch composition noise $N_i(\mathcal{B}_t, t)$ relative to sample i is:

$$N_i(\mathcal{B}_t, t) \coloneqq \frac{1}{B} \sum_{j \in \mathcal{B}_t, j \neq i} l_j(t). \tag{4}$$

This term represents the average loss of the other $B - 1$ samples accompanying i in batch $\mathcal{B}_t$.

In particular, $N_i(\mathcal{B}_t, t)$ varies stochastically across different steps t where i is selected, primarily due to the random sampling of the set $\mathcal{B}_t \setminus \{i\}$. The sequence of these noise terms for sample i constitutes a major source of high-frequency fluctuation that the EMA aims to filter.

4.2. EMA as a Filtering Mechanism

Let $i \in \mathcal{D}$, $\{t_k\}_{k \in \mathbb{N}}$ denote the sequence of training steps where i is included in the sampled batch ($i \in \mathcal{B}_{t_k}$), and $\mathcal{L}_i = \{L(\mathcal{B}_{t_k}, t_k)\}_{k=1}^{\infty}$ be the sequence of batch losses observed by sample i. Based on Equation (3), we define the corresponding sequences for the scaled signal and noise: $\mathcal{S}_i = \{\frac{1}{B} l_i(t_k)\}_{k=1}^{\infty}$ and $\mathcal{N}_i = \{N_i(\mathcal{B}_{t_k}, t_k)\}_{k=1}^{\infty}$. By definition, $\mathcal{L}_i[k] = \mathcal{S}_i[k] + \mathcal{N}_i[k]$ for all $k \geq 1$.

The BLS update rule (Equation (2)) defines a discrete-time filtering operation on the subsequence of steps $\{t_k\}$. Let $s_i^{(k)} \coloneqq s_i(t_k)$ be the score of sample i immediately after its k-th update (at step t_k). Ignoring the precise global timing t_k and focusing on the update index k for sample i, then:

$$s_i^{(k)} = \alpha s_i^{(k-1)} + (1-\alpha) \mathcal{L}_i[k], \quad k \geq 1 \tag{5}$$

with initial condition $s_i^{(0)} = s_i(t_0)$ (the score before the first update at t_1). This is the standard form of a first-order infinite impulse response (IIR) filter, representing a linear time-invariant (LTI) system [32] operating on the observation sequence $\mathcal{L}_i$.

Assumption 4.2 (Frequency Separation Hypothesis)**.** The stochastic fluctuations in the batch composition noise sequence $\mathcal{N}_i$, primarily driven by the random sampling of batch members, possess significant spectral energy at higher frequencies compared to the typical evolution of the scaled per-sample loss sequence $\mathcal{S}_i$. The latter reflects the comparatively slower changes in the model's performance on sample i due to gradual parameter updates $\boldsymbol{\theta}_t$.

Remark 4.3. This assumption is grounded in the differing timescales of batch sampling (per-step randomness) and model learning (changes over multiple updates or epochs). While $l_i(t)$ can change, its persistent trend is usually smoother than the noise induced by constantly varying batch compositions.

Proposition 4.4 (BLS Score as Low-Pass Filtered Estimate)**.** *The BLS score $s_i^{(k)}$ derived from Equation* (5) *is the output of applying a first-order IIR low-pass filter H_α to the input sequence $\mathcal{L}_i = \mathcal{S}_i + \mathcal{N}_i$, plus a term decaying with the initial condition $s_i^{(0)}$:*

$$s_i^{(k)} = (H_\alpha * \mathcal{L}_i)[k] + \alpha^k s_i^{(0)} \tag{6}$$

where $*$ denotes discrete convolution and H_α is the filter with impulse response $h[n] = (1-\alpha)\alpha^n u[n]$ ($u[n]$ being the Heaviside step function). By filter linearity, this decomposes into filtered signal and noise components:

$$s_i^{(k)} = \underbrace{(H_\alpha * \mathcal{S}_i)[k]}_{\textit{Smoothed Signal Trend}} + \underbrace{(H_\alpha * \mathcal{N}_i)[k]}_{\textit{Smoothed Noise Baseline}} + \alpha^k s_i^{(0)}. \tag{7}$$

Invoking Assumption 4.2, the low-pass characteristic of H_α ensures that high-frequency components, predominantly present in the noise $\mathcal{N}_i$, are attenuated in $(H_\alpha * \mathcal{N}_i)[k]$. Consequently, $s_i^{(k)}$ provides an estimate emphasizing the smoothed, persistent trend of the scaled per-sample loss $\mathcal{S}_i$, with reduced influence from stochastic batch variations.

Proof. The update rule Equation (5), $s_i^{(k)} = \alpha s_i^{(k-1)} + (1-\alpha)\mathcal{L}_i[k]$, defines the output $s_i^{(k)}$ of a discrete-time LTI system [32] with input sequence $\mathcal{L}_i[k] = L(\mathcal{B}_{t_k}, t_k)$. The impulse response of this first-order recursive system is $h[n] = (1-\alpha)\alpha^n u[n]$, where $u[n]$ is the Heaviside step function.

For a general input $\mathcal{L}_i$, the output of the LTI system is given by the convolution of the input with the impulse response, plus the zero-input response due to the initial condition:

$$\begin{aligned} s_i^{(k)} =& \sum_{j=0}^{k} h[j]\mathcal{L}_i[k-j] + \alpha^k s_i^{(0)} \\ =& (H_\alpha * \mathcal{L}_i)[k] + \alpha^k s_i^{(0)}, \end{aligned} \tag{8}$$

where H_α denotes the filter with impulse response $h[n]$ and $*$ denotes discrete convolution. This confirms Equation (6). The filter H_α is a standard first-order IIR low-pass filter. Its frequency response is $H(e^{j\omega}) = \frac{1-\alpha}{1-\alpha e^{-j\omega}}$. The magnitude $|H(e^{j\omega})| = \frac{1-\alpha}{\sqrt{1-2\alpha\cos(\omega)+\alpha^2}}$ is maximized at $\omega = 0$ (DC) and decreases monotonically as frequency $|\omega|$ increases towards π. This low-pass characteristic means the filter attenuates high-frequency components while passing low-frequency components. The parameter $\alpha \in (0,1)$ controls the cutoff frequency; $\alpha \to 1$ implies stronger smoothing (lower cutoff).

By the linearity property of convolution (and the filter H_α), applying the filter to the decomposed input $\mathcal{L}_i = \mathcal{S}_i + \mathcal{N}_i$ (from Equation (3)) yields:

$$\begin{aligned} (H_\alpha * \mathcal{L}_i)[k] =& (H_\alpha * (\mathcal{S}_i + \mathcal{N}_i))[k] \\ =& (H_\alpha * \mathcal{S}_i)[k] + (H_\alpha * \mathcal{N}_i)[k]. \end{aligned} \tag{9}$$

Substituting this into the expression for $s_i^{(k)}$ gives the decomposition in Equation (7).

Invoking Assumption 4.2, the batch composition noise sequence $\mathcal{N}_i$ is assumed to have significant energy at higher frequencies compared to the scaled per-sample loss sequence $\mathcal{S}_i$. The low-pass filter H_α therefore primarily: Passes and smooths the lower-frequency content of $\mathcal{S}_i$, resulting in $(H_\alpha * \mathcal{S}_i)[k]$ which reflects the persistent trend of the scaled sample loss. Attenuates the higher-frequency content dominant in $\mathcal{N}_i$, resulting in a smoothed noise term $(H_\alpha * \mathcal{N}_i)[k]$ with reduced variance. Consequently, the BLS score $s_i^{(k)}$ predominantly captures the smoothed signal trend, effectively filtering out much of the noise induced by random batch composition. This allows $s_i^{(k)}$ to serve as a robust estimate of the sample's persistent loss contribution over time. □

4.3. Discussion Effectiveness of BLS

The theoretical analysis, culminating in Proposition 4.4, offers several insights into BLS's efficacy as a proxy for sample importance:

Sample-Specific Filtered History. The conditional EMA update (Equation (2)) ensures each sample score, $s_i(t)$, aggregates information solely from batch losses observed during its active participation. Applying the filter H_α to this unique sequence generates a sample-specific history, distinguishing $s_i(t)$ from $s_j(t)$ for $i \neq j$.

Filtering Batch Composition Randomness. BLS's core mechanism is noise filtering. The EMA acts as a low-pass filter H_α which, under Assumption 4.2, effectively attenuates high-frequency variations inherent in the batch composition noise $\mathcal{N}_i$. This prevents scores from being skewed by transient, unrepresentative losses of randomly co-occurring samples within a single batch.

Approximating Persistent Loss Contribution. The BLS score (Equation (7)) approximates the sum of the smoothed scaled signal $(H_\alpha * \mathcal{S}_i)[k]$ and smoothed noise $(H_\alpha * \mathcal{N}_i)[k]$. The term $(H_\alpha * \mathcal{S}_i)[k]$ captures the smoothed historical trend of the sample's scaled loss contribution $(1/B)l_i(t)$. If the smoothed noise $(H_\alpha * \mathcal{N}_i)[k]$ acts as a relatively stable baseline, score differences $s_i(t) - s_j(t)$ are primarily driven by differences in their smoothed signal components. Thus, BLS effectively ranks samples by their persistent, time-averaged contribution to batch loss.

Role of EMA Parameter α. The decay factor α governs the filter's characteristics. A larger α (closer to 1) implies stronger smoothing (lower cutoff frequency), leading to greater noise attenuation but reduced responsiveness to true changes in sample importance $l_i(t)$. Conversely, smaller α values yield more responsive but noisier scores. Optimal α selection balances noise suppression with tracking sample importance dynamics.

Table 1. BLS effectively proxies per-sample loss methods (InfoBatch, SeTa) on large-scale datasets. Results show comparable or improved performance for BLS-variants across diverse vision-language and scene text recognition tasks. (a) ToCa dataset (zero-shot captioning, 3M samples). (b) MJ+ST dataset (scene text recognition, 15M samples). (c) SS1M dataset (cross-domain captioning, 3M samples). Pruned % indicates data reduction.

(a) ViECap on ToCa

Method	Pruned %	NoCaps Val (CIDEr)				COCO
		In	Near	Out	Overall	Test
Full	-	63.2	68.8	70.2	70.5	95.2
InfoBatch	23.6	63.4	68.2	70.4	70.2	94.4
SeTa	31.7	63.4	69.7	71.4	71.5	95.3
BLS-InfoBatch	23.8	65.1	68.9	69.8	70.6	94.7
BLS-SeTa	32.0	64.3	69.4	71.0	71.2	95.1

(b) ABINet on MJ+ST

Pruned %	IIIT5k	SVT	IC15	SVTP	CUTE80
	Accuracy %				
-	96.1	93.4	85.4	88.7	89.2
26.6	96.3	93.6	85.4	88.8	89.4
28.1	96.2	93.4	85.6	89.3	89.2
25.4	96.1	93.8	85.5	89.1	89.9
33.0	96.2	94.0	85.5	88.7	89.1

(c) ViECap on SS1M

Pruned %	COCO		Flickr30k	
	B@4	C	B@4	C
-	9.6	45.1	6.5	22.3
23.0	9.2	44.4	6.4	22.4
40.0	9.5	45.6	6.4	22.0
24.9	9.3	44.5	6.5	22.5
41.1	9.6	45.5	6.5	22.2

Table 2. Comparison of state-of-the-art methods on CIFAR classification (ResNet18, Accuracy %) at different pruning rates. BLS effectively proxies per-sample loss methods to achieve comparable accuracy to original methods under the same pruning rate. †: adjusted epochs in original paper. ‡: same epochs as other methods. \: unattainable ratio. Random*: dynamic random. Performance relative to Full training is indicated (↓: drop, ↑: gain).

Method	CIFAR10			CIFAR100		
	30%	50%	70%	30%	50%	70%
ResNet18	95.6			78.2		
Random	94.6 $_{\downarrow 1.0}$	93.3 $_{\downarrow 2.3}$	90.2 $_{\downarrow 5.4}$	73.8 $_{\downarrow 4.4}$	72.1 $_{\downarrow 6.1}$	69.7 $_{\downarrow 8.5}$
GraNd-4 [34]	95.3 $_{\downarrow 0.3}$	94.6 $_{\downarrow 1.0}$	91.2 $_{\downarrow 4.4}$	74.6 $_{\downarrow 3.6}$	71.4 $_{\downarrow 6.8}$	68.8 $_{\downarrow 9.4}$
EL2N-20 [45]	95.3 $_{\downarrow 0.3}$	95.1 $_{\downarrow 0.5}$	91.9 $_{\downarrow 3.7}$	77.2 $_{\downarrow 1.0}$	72.1 $_{\downarrow 6.1}$	-
DP [50]	94.9 $_{\downarrow 0.7}$	93.8 $_{\downarrow 1.8}$	90.8 $_{\downarrow 4.8}$	77.2 $_{\downarrow 1.0}$	73.1 $_{\downarrow 5.1}$	-
Random*	94.8 $_{\downarrow 0.8}$	94.5 $_{\downarrow 1.1}$	93.0 $_{\downarrow 2.6}$	77.3 $_{\downarrow 0.9}$	75.3 $_{\downarrow 2.9}$	-
ϵ-greedy [38]	95.2 $_{\downarrow 0.4}$	94.9 $_{\downarrow 0.7}$	94.1 $_{\downarrow 1.5}$	76.4 $_{\downarrow 1.8}$	74.8 $_{\downarrow 3.4}$	-
UCB [38]	95.3 $_{\downarrow 0.3}$	94.7 $_{\downarrow 0.9}$	93.9 $_{\downarrow 1.7}$	77.3 $_{\downarrow 0.9}$	75.3 $_{\downarrow 2.9}$	-
InfoBatch† [36]	95.6 $_{\uparrow 0.0}$	95.1 $_{\downarrow 0.5}$	94.7 $_{\downarrow 0.9}$	78.2 $_{\uparrow 0.0}$	78.1 $_{\downarrow 0.1}$	76.5 $_{\downarrow 1.7}$
InfoBatch‡ [36]	95.6 $_{\uparrow 0.0}$	95.0 $_{\downarrow 0.6}$	94.4 $_{\downarrow 1.2}$	78.3 $_{\uparrow 0.1}$	77.7 $_{\downarrow 0.5}$	\
SeTa [54]	95.7 $_{\uparrow 0.1}$	95.3 $_{\downarrow 0.3}$	95.0 $_{\downarrow 0.6}$	78.4 $_{\uparrow 0.2}$	78.0 $_{\downarrow 0.2}$	76.7 $_{\downarrow 1.6}$
BLS-InfoBatch	95.6 $_{\uparrow 0.0}$	95.1 $_{\downarrow 0.5}$	94.5 $_{\downarrow 1.1}$	78.4 $_{\uparrow 0.2}$	77.7 $_{\downarrow 0.5}$	\
BLS-SeTa	95.6 $_{\uparrow 0.0}$	95.3 $_{\downarrow 0.3}$	95.0 $_{\downarrow 0.6}$	78.5 $_{\uparrow 0.3}$	78.1 $_{\downarrow 0.1}$	76.7 $_{\downarrow 1.5}$

5. Experiments

We conduct extensive experiments, that include 14 datasets, 11 tasks, 18 model architectures and 2 pruning methods, to evaluate the *effectiveness*, *efficiency*, and *generalization* capabilities of the proposed BLS. Our evaluation strategy is designed to progressively demonstrate BLS's capabilities: first establishing its validity as an effective proxy for per-sample loss in standard settings, then showcasing its competitive performance and broad generalization across challenging tasks and architectures, and finally, empirically validating its underlying theoretical mechanism.

Tasks and Datasets. Evaluation spans classification (CIFAR [25], ImageNet [40]), object detection/segmentation (COCO [6]), vision-language (image/video captioning on COCO [7]/NoCaps [1]/SS1M [14]/ToCa [55]/MSR-VTT [48]), scene text recognition on MJ+ST ([15, 22]), image generation (MNIST [9], CIFAR10), multi-view stereo (WHU-MVS [27]), and semi-supervised learning [46] (image classification on CIFAR100, text classification on Yelp Review [51], audio classification on ESC-50 [35]).

Architectures. We test across CNNs (ResNet18/50 [16], EfficientNet [44], YOLOv5 [39]), Transformers (ViT [10], Swin [28]), and emerging architectures Vim [56]. Vision-language tasks use ViECap [13] or ABINet [12] backbones. Generative models include VAE [24] and DDPM [18, 19]. MVS uses Ada-MVS [26]. Semi-supervised learning uses FixMatch [43]/FlexMatch [52]/Dash [49].

BLS Integration. We evaluate BLS by integrating it into two representative per-sample loss-based pruning methods: InfoBatch [36] and SeTa [54], denoted as BLS-InfoBatch and BLS-SeTa. This allows direct assessment of BLS as a drop-in replacement for $l_i(t)$ scoring; such integration is achieved with the minimal ***one-line proxy*** and ***three-line injection*** mechanisms detailed in Suppl. Sec. 7. To ensure fairness, all experiments adopt the established settings corresponding to their respective tasks, models, and datasets.

Evaluation Metrics. Model performance is measured using standard task-specific metrics. Efficiency is primarily evaluated by the percentage of data pruned **(Pruned %)** while maintaining comparable performance to the baseline or original method. We justify this choice in Suppl. Sec. 9.

5.1. Effective Proxy for Per-Sample Loss

We first validate BLS's ability to serve as an effective proxy by integrating it into established per-sample loss-based pruning methods, InfoBatch [36] and SeTa, and comparing their performance with their BLS-enhanced counterparts (BLS-InfoBatch and BLS-SeTa).

Table 1 presents these critical comparisons across three challenging large-scale vision-language and scene text recognition benchmarks. On the ToCa dataset for visual captioning (Table 1a), BLS-InfoBatch and BLS-SeTa achieve nearly identical or even slightly improved CIDEr scores (e.g., BLS-InfoBatch Overall CIDEr 70.6 vs. InfoBatch 70.2; BLS-

Table 3. BLS demonstrates strong cross-architecture generalization on ImageNet-1K and CIFAR100 with CNNs, Transformers, and Mambas. Format: Accuracy / Pruned %.

Method	ImageNet-1K						CIFAR100	
	CNN			Transformer		Mamba		
	R18	R50	EfficientNet	ViT	Swin	Vim	R18	R50
Full	69.5	78.6	76.1	73.3	80.0	75.7	78.2	80.6
BLS-InfoBatch	69.4 / 32.8	78.2 / 31.5	75.7 / 32.8	73.2 / 26.2	80.0 / 30.8	75.6 / 22.9	78.3 / 33.9	80.5 / 38.6
BLS-SeTa	69.5 / 31.3	78.4 / 32.3	76.0 / 31.1	73.3 / 25.2	80.0 / 44.5	75.6 / 29.4	78.4 / 40.1	80.7 / 40.7

Table 4. Consolidated results for various methods on different datasets and tasks. (a) ViECap on COCO dataset: Image captioning performance with 556K samples. (b) ViECap on MSR-VTT dataset: Video captioning performance with 180K samples. (c) Ada-MVS on WHU-MVS dataset: Multi-view stereo performance with 28K image-depth map samples.

(a) Image Captioning

Method	Pruned %	NoCaps Val (CIDEr)				COCO			
		In	Near	Out	Overall	B@4	M	C	S
Full	-	58.4	63.1	65.3	65.2	27.1	24.6	91.5	18.0
BLS-InfoBatch	20.9	58.8	63.1	65.8	65.3	27.1	24.6	91.9	17.9
BLS-SeTa	28.1	58.9	63.1	65.6	65.3	27.2	24.7	92.0	18.0

(b) Video Captioning

Pruned %	MSR-VTT			
	B@4	M	C	S
-	23.4	20.8	27.9	5.0
21.8	24.7	21.0	29.9	5.0
40.3	24.7	21.3	29.3	4.9

(c) Multi-View Stereo

Pruned %	WHU-MVS	
	<3-i	<0.6m
-	95.01	97.38
45.0	95.17	97.33
37.5	95.04	97.45

SeTa 71.2 vs. SeTa 71.5) while maintaining comparable data pruning percentages (23.8% and 30.0% respectively). Similarly, for scene text recognition on the 15M-sample MJ+ST dataset (Table 1b), BLS-variants demonstrate strong parity. For instance, BLS-SeTa (pruning 33.0%) achieves an accuracy of 94.0% on SVT, matching or exceeding the original SeTa (93.4% at 28.1% pruning). The trend continues on the SS1M dataset for cross-domain image captioning (Table 1c), where BLS-SeTa (41.1% pruned) yields a COCO CIDEr of 45.5, on par with SeTa (45.6 at 40.0% pruned). These results across diverse, large-scale datasets with complex underlying losses strongly suggest that BLS effectively captures the essential sample importance signals that these sophisticated per-sample loss methods rely upon.

Further corroborating these findings, Table 2 details performance on standard dynamic data pruning benchmarks, CIFAR10 and CIFAR100, using ResNet18. When compared directly with InfoBatch and SeTa, the BLS-integrated versions consistently deliver statistically indistinguishable accuracies across various pruning rates (30%, 50%, 70%). Critically, both BLS-InfoBatch and BLS-SeTa often achieve performance on par with the fully trained baseline at moderate pruning rates. This not only confirms BLS's efficacy as a proxy but also hints at its potential to regularize training by focusing on more informative samples.

5.2. Broad Generalization of Batch Loss Signal

A cornerstone of BLS's utility is its inherent generalizability, stemming from its sole reliance on the ubiquitous mean batch loss signal. This independence from specific loss or architectural details allows broad applicability, which we demonstrate across diverse models and tasks.

Table 5. Image generation with different architectures (VAE on MNIST, DDPM on CIFAR10, DDPM with Classifier Guidance (CFG) on CIFAR10). FID: Frechet Inception Distance.

Method	VAE		DDPM		DDPM-CFG	
	Pruned %	FID	Pruned %	FID	Pruned %	FID
Full	-	35.34	-	16.38	-	14.89
BLS-InfoBatch	28.9	35.55	28.7	17.01	22.0	14.99
BLS-SeTa	34.0	35.34	30.8	16.35	22.3	14.72

Table 6. Semi-supervised learning including image classification (FixMatch on CIFAR100), text sentiment classification (FlexMatch on Yelp Review) and audio classification (Dash on ESC-50).

Method	Image		Text		Audio	
	Pruned %	Acc	Pruned %	Acc	Pruned %	Acc
Full	-	61.9	-	53.7	-	64.5
BLS-InfoBatch	31.3	61.9	38.1	54.4	25.0	64.5
BLS-SeTa	47.4	61.9	49.4	54.7	28.7	64.6

Architectural Agnosticism. BLS exhibits robust performance across varied architectures. Table 3 shows that for ImageNet-1K and CIFAR100, BLS-guided pruning maintains accuracy close to full training across CNNs (ResNets, EfficientNet), Transformers (ViT, Swin), and even Mamba (Vim). For instance, Vim with BLS-InfoBatch on ImageNet-1K prunes 22.9% data while nearly matching full accuracy (75.6% vs. 75.7%). This efficacy across fundamentally different architectures underscores BLS's versatility, requiring no architecture-specific adaptations.

Effectiveness in Complex Vision Tasks. BLS navi-

Table 7. Enhancing computational efficiency of a unified YOLOv5 framework for image classification on CIFAR100, alongside object detection and instance segmentation on COCO, through BLS.

Method	Image Classification			Object Detection			Instance Segmentation		
	Pruned %	Acc@1	Acc@5	Pruned %	$mAP_{.5}$	$mAP_{.5:.95}$	Pruned %	$mAP_{.5}$	$mAP_{.5:.95}$
YOLOv5n	-	58.0	84.6	-	38.2	22.1	-	29.5	16.3
BLS-InfoBatch	20.4	57.6	84.0	21.6	38.5	22.2	29.5	31.0	17.2
BLS-SeTa	35.4	58.0	84.6	25.2	38.3	22.1	36.1	31.7	17.6

gates tasks with multi-component losses, where per-sample loss $l_i(t)$ is arduous to define or access (Sec. 8). Table 7 (YOLOv5) shows BLS-variants maintain or improve mAP for object detection and instance segmentation while pruning significantly (e.g., BLS-SeTa +2.2 $mAP_{.5}$ for segmentation with 36.1% pruning). Similarly, for Multi-View Stereo (Table 4c, WHU-MVS), BLS-SeTa prunes 37.5% data with slight accuracy gains. These results highlight BLS's advantage in bypassing intricate loss disentanglement.

Generalization to Vision-Language (VL) Scenarios. BLS robustly extends to VL tasks involving complex cross-modal modeling and large-scale, potentially noisy datasets. Results on image/video captioning (Table 1, Table 4a-b) show BLS-variants maintaining or improving metrics like CIDEr and B@4 (e.g., COCO captioning CIDEr 92.0 with BLS-SeTa vs. 91.5 baseline, pruning 28.1%) while pruning 20-40% of data. This underscores BLS's utility without needing specialized per-sample loss definitions for intricate multimodal signals.

Adaptability to Diverse Learning Paradigms. Beyond supervised recognition and VL tasks, BLS demonstrates compatibility with fundamentally different learning approaches. *Generative Modeling (Table 5):* For VAEs and Diffusion Models (DDPM/DDPM-CFG), BLS-guided pruning (22-34%) preserves FID scores, indicating high-quality generation with reduced data. *Semi-Supervised Learning (SSL, Table 6):* BLS readily integrates into SSL for image, text, and audio classification, enabling substantial data pruning (25-49%) without degrading accuracy. This is notable as SSL consistency losses often defy traditional hardness metric decomposition.

This extensive empirical validation across diverse tasks, architectures, complex losses, and learning paradigms robustly confirms that BLS's reliance on the universal mean batch loss grants it exceptional generalization. It stands as a highly practical and versatile tool for enhancing training efficiency across the machine learning landscape.

5.3. Empirical Validation of Frequency Separation

To empirically validate our Frequency Separation Hypothesis (Assumption 4.2), we analyzed the spectral characteristics of the scaled signal $\mathcal{S}_i$ and batch composition noise $\mathcal{N}_i$ components derived from the mean batch loss decomposition

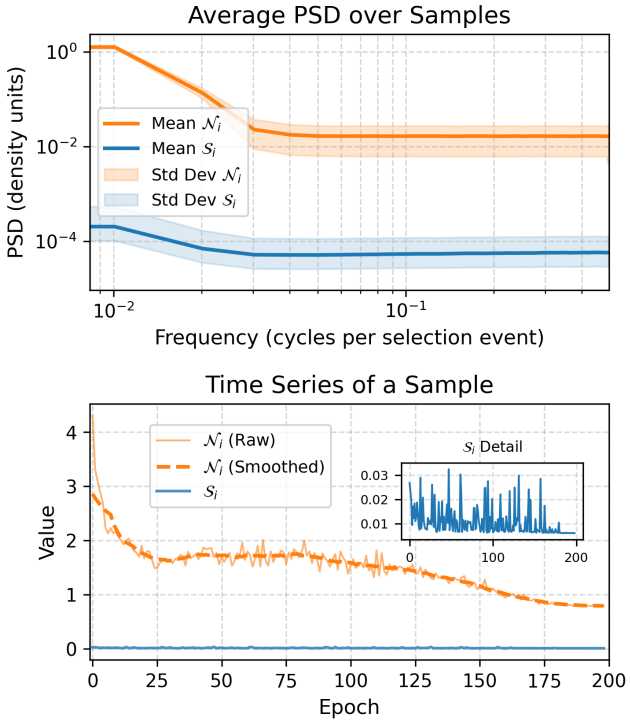

Figure 2. Empirical validation of frequency separation. **Top:** Average PSD for scaled signal (Mean $\mathcal{S}_i$) and batch noise (Mean $\mathcal{N}_i$). **Bottom:** Time series for a representative sample. $\mathcal{S}_i$ magnitude is much smaller (see inset).

(Eq. 3). We computed the Power Spectral Density (PSD) using Welch's method [47] for sequences collected during standard training (ResNet18 on CIFAR100).

Figure 2 presents the average PSD over 50K samples and 200 epochs and a representative time series. The results provide strong empirical support for our hypothesis. The average PSD (Figure 2, top) demonstrates a clear separation: the noise component $\mathcal{N}_i$ (orange) possesses significantly higher energy density than the scaled signal $\mathcal{S}_i$ (blue) across all frequencies, with the disparity being particularly pronounced at higher frequencies. While $\mathcal{N}_i$ retains substantial high-frequency content, characteristic of stochastic batch sampling, $\mathcal{S}_i$'s energy is concentrated at lower frequencies, reflecting slower model dynamics. The time series

plot (Figure 2, bottom) visually confirms this, showing the high volatility of $\mathcal{N}_i$ around its trend compared to the small magnitude and smoother evolution of $\mathcal{S}_i$.

This observed frequency separation empirically justifies the core mechanism of BLS. The inherent low-pass filtering characteristic of the EMA update (Proposition 4.4) effectively attenuates the dominant, high-frequency noise $\mathcal{N}_i$, enabling the BLS score $s_i(t)$ to capture the persistent, low-frequency signal trend related to $\mathcal{S}_i$. This validates using BLS, derived solely from batch loss observations, as a efficient proxy for identifying samples with consistently high loss contributions.

5.4. Ablation and Hyperparameter Analysis

Table 8. Ablation study of BLS components on CIFAR100 (ResNet18/50). "P %" denotes Pruning Percentage. "Time" includes pruning cost and training time. "+ BLS" refers to the full BLS integrated method. "+ w/o EMA BLS" uses only the last batch loss for scoring active samples.

Method	R18			R50		
	Acc	Time	P %	Acc	Time	P %
Full	78.2	2.46h	-	80.6	4.10h	-
InfoBatch	78.2	1.63h	33.8	80.6	2.74h	33.0
+ w/o EMA BLS	77.9	1.65h	32.8	79.9	2.75h	32.8
+ BLS	**78.3**	1.63h	33.9	**80.6**	2.74h	33.1
SeTa	78.3	1.53h	40.0	80.7	2.53h	39.8
+ w/o EMA BLS	78.0	1.53h	39.8	79.9	2.55h	38.8
+ BLS	**78.4**	1.52h	40.1	**80.8**	2.52h	40.1

Ablation Analysis. We ablate BLS components on CIFAR100 (ResNet18/50), with results in Table 8. We compare against full training, original per-sample loss methods (InfoBatch, SeTa), BLS using only the most recent batch loss for active samples ("+ w/o EMA BLS"), and full BLS integration ("+ BLS"). The data underscores the criticality of the EMA for temporal smoothing. Removing EMA ("+ w/o EMA BLS") consistently degrades accuracy compared to full BLS across both InfoBatch and SeTa integrations (e.g., R18 InfoBatch: 77.9% vs. 78.3%; R50 SeTa: 79.9% vs. 80.8%). This highlights that instantaneous batch losses alone are insufficient, and EMA's noise filtering is essential for robust scoring. Furthermore, full BLS-integrated methods ("+ BLS") achieve accuracies on par with, or slightly exceeding, their original per-sample loss counterparts, while maintaining similar data pruning percentages (P %). For instance, BLS-SeTa on R50 matches SeTa's accuracy (80.7% vs. 80.8%) with comparable pruning ($\sim$40%). This validates BLS's complete mechanism. Notably, the substantial data pruning achieved (e.g., 30-40%) directly translates to reduced training time (Time column), establishing Pruned % as a reliable proxy for computational savings, independent of hardware-specific timing variations.

Hyperparameter Analysis. Figure 3 illustrates BLS's

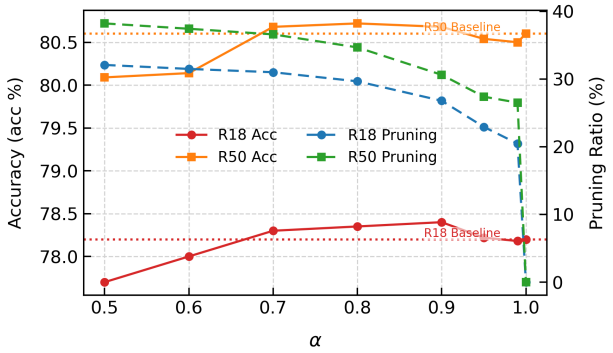

Figure 3. Effect of EMA decay factor α. Accuracy and Pruning Ratio are shown for ResNet18 and ResNet50.

sensitivity to the EMA decay factor α on CIFAR100 (ResNet18/50). For $\alpha \in [0.5, 0.6]$, scores are highly responsive, yielding substantial pruning ratios (R18: $\sim$31-33%, R50: $\sim$35-37%) but accuracies (R18: $\sim$79.0-79.1%, R50: $\sim$80.1-80.2%) are slightly below their respective baselines, likely due to noise sensitivity. As α increases to the range $[0.7, 0.8]$, a favorable trade-off is observed. ResNet18 accuracy peaks at 79.4% (e.g., $\alpha = 0.7, 0.8$) with a pruning ratio of $\sim$30-31%. ResNet50 accuracy also peaks at 80.7% ($\alpha = 0.8$) with $\sim$30% pruning, surpassing its baseline. This region suggests effective noise filtering while maintaining responsiveness. For $\alpha \in [0.9, 1.0)$, pruning ratios decline sharply for both models as scores become overly smoothed and static, with $\alpha = 1.0$ effectively replicating full training (near-zero pruning and baseline accuracy). This analysis indicates that α values between 0.7 and 0.8 offer a robust balance of high accuracy and significant pruning. Consequently, we default to $\alpha = 0.7$ for other experiments.

6. Conclusion

This paper introduces the BLS, a practical and efficient method for dynamic data pruning that uniquely infers sample importance solely from readily available mean batch losses, bypassing the complexities of accessing per-sample loss. By applying a sample-specific Exponential Moving Average to the sequence of batch losses encountered by each sample, BLS effectively filters noise arising from stochastic batch composition, as rigorously demonstrated by our signal-processing analysis. Extensive empirical validation across ten diverse tasks, architectures, and learning paradigms confirms that BLS enables significant data pruning while maintaining performance comparable to state-of-the-art methods reliant on direct per-sample losses. Its negligible overhead and implementation simplicity make BLS a broadly applicable tool for enhancing various training tasks.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant 62471394, U21B2041, 62306241 and 62576284.

References

[1] H. Agrawal, K. Desai, Y. Wang, X. Chen, R. Jain, M. Johnson, D. Batra, D. Parikh, S. Lee, and P. Anderson. Nocaps: Novel object captioning at scale. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8948–8957, 2019. 5

[2] G. Alain, A. Lamb, C. Sankar, A. Courville, and Y. Bengio. Variance reduction in sgd by distributed importance sampling. *arXiv preprint arXiv:1511.06481*, 2015. 2

[3] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009. 2

[4] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020. 1

[5] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. volume 40, pages 834–848. IEEE, 2017. 1

[6] X. Chen, H. Fang, T.-Y. Lin, R. Vedantam, S. Gupta, P. Dollár, and C. L. Zitnick. Microsoft coco captions: Data collection and evaluation server. *arXiv preprint arXiv:1504.00325*, 2015. 1, 5

[7] X. Chen, H. Fang, T.-Y. Lin, R. Vedantam, S. Gupta, P. Dollár, and C. L. Zitnick. Microsoft coco captions: Data collection and evaluation server. *arXiv preprint arXiv:1504.00325*, 2015. 5

[8] C. Coleman, C. Yeh, S. Mussmann, B. Mirzasoleiman, P. Bailis, P. Liang, J. Leskovec, and M. Zaharia. Selection via proxy: Efficient data selection for deep learning. In *International Conference on Learning Representations*, 2020. 1, 2

[9] L. Deng. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012. 5

[10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 1, 5

[11] T. Evans, S. Pathak, H. Merzic, J. Schwarz, R. Tanno, and O. J. Hénaff. Bad students make great teachers: Active learning accelerates large-scale visual understanding. In *Proceedings of the 18th European Conference on Computer Vision*, volume 15075, pages 264–280. Springer, 2024. 4

[12] S. Fang, H. Xie, Y. Wang, Z. Mao, and Y. Zhang. Read like humans: Autonomous, bidirectional and iterative language modeling for scene text recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7094–7103, 2021. 5

[13] J. Fei, T. Wang, J. Zhang, Z. He, C. Wang, and F. Zheng. Transferable decoding with visual entities for zero-shot image captioning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3136–3146, 2023. 5

[14] Y. Feng, L. Ma, W. Liu, and J. Luo. Unsupervised image captioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4125–4134, 2019. 5

[15] A. Gupta, A. Vedaldi, and A. Zisserman. Synthetic data for text localisation in natural images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2315–2324, 2016. 5

[16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 5

[17] M. He, S. Yang, T. Huang, and B. Zhao. Large-scale dataset pruning with dynamic uncertainty. pages 7713–7722, 2024. 1

[18] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 5

[19] J. Ho and T. Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 5

[20] S. Huang, H. Zhang, and X. Li. Enhance vision-language alignment with noise. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 17449–17457, 2025. 1

[21] R. Iyer, N. Khargoankar, J. Bilmes, and H. Asanani. Submodular combinatorial information measures with applications in machine learning. In *Proceedings of the 34th Annual Conference on Algorithmic Learning Theory*, pages 722–754, 2021. 2

[22] M. Jaderberg, K. Simonyan, A. Vedaldi, and A. Zisserman. Synthetic data and artificial neural networks for natural scene text recognition. *ArXiv*, abs/1406.2227, 2014. 5

[23] A. Katharopoulos and F. Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *International conference on machine learning*, pages 2538–2547. PMLR, 2018. 2

[24] D. P. Kingma and M. Welling. Auto-encoding variational bayes. In Y. Bengio and Y. LeCun, editors, *International Conference on Learning Representations*, 2014. 5

[25] A. Krizhevsky, V. Nair, and G. Hinton. Cifar-100 (canadian institute for advanced research). 5

[26] J. Liu, J. Gao, S. Ji, C. Zeng, S. Zhang, and J. Gong. Deep learning based multi-view stereo matching and 3d scene reconstruction from oblique aerial images. *ISPRS Journal of Photogrammetry and Remote Sensing*, 204:42–60, 2023. 5

[27] J. Liu and S. Ji. A novel recurrent encoder-decoder structure for large-scale multi-view stereo reconstruction from an open aerial dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6050–6059, 2020. 5

[28] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 5

[29] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015. 1

[30] I. Loshchilov and F. Hutter. Online batch selection for faster training of neural networks. In *International Conference on Learning Representations*, 2016. 1, 2

[31] S. Mindermann, I. Babuschkin, J. DWI Setyawan, J. W. Rae, M. Warkentin, N. Savinov, O. Vinyals, R. Hadsell, M. Vladymyrov, and J. Grau-Moya. Prioritized training on points that are learnable, worth learning, and not yet learned. In *International Conference on Machine Learning*, pages 15703–15731. PMLR, 2022. 1, 2

[32] A. V. Oppenheim. *Discrete-time signal processing*. Pearson Education India, 1999. 3, 4

[33] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, volume 32, 2019. 1

[34] M. Paul, S. Ganguli, and G. K. Dziugaite. Deep learning on a data diet: Finding important examples early in training. *Advances in neural information processing systems*, 34:20596–20607, 2021. 1, 2, 5

[35] K. J. Piczak. Esc: Dataset for environmental sound classification. In *Proceedings of the 23rd ACM international conference on Multimedia*, pages 1015–1018, 2015. 5

[36] Z. Qin, K. Wang, Z. Zheng, J. Gu, X. Peng, xu Zhao Pan, D. Zhou, L. Shang, B. Sun, X. Xie, and Y. You. Infobatch: Lossless training speed up by unbiased dynamic data pruning. In *The Twelfth International Conference on Learning Representations*, 2024. 1, 2, 5, 4

[37] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. 1

[38] R. S. Raju, K. Daruwalla, and M. Lipasti. Accelerating deep learning with dynamic data pruning. *arXiv preprint arXiv:2111.12621*, 2021. 1, 5

[39] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016. 1, 5, 2, 3

[40] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. S. Bernstein, A. C. Berg, and L. Fei-Fei. Imagenet large scale visual recognition challenge. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, volume 115, pages 211–252, 2015. 1, 5

[41] O. Sener and S. Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations*, 2018. 2

[42] S. L. Smith, P.-J. Kindermans, C. Ying, and Q. V. Le. Don't decay the learning rate, increase the batch size. *arXiv preprint arXiv:1711.00489*, 2017. 2

[43] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *Advances in neural information processing systems*, 33:596–608, 2020. 5

[44] M. Tan and Q. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114, 2019. 5

[45] M. Toneva, A. Sordoni, A. V. Comport, A. Trischler, Y. Bengio, and G. J. Gordon. An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations*, 2019. 2, 5

[46] Y. Wang, H. Chen, Y. Fan, W. Sun, R. Tao, W. Hou, R. Wang, L. Yang, Z. Zhou, L.-Z. Guo, et al. Usb: A unified semi-supervised learning benchmark for classification. *Advances in Neural Information Processing Systems*, 35:3938–3961, 2022. 1, 5, 3

[47] P. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967. 7

[48] J. Xu, T. Mei, T. Yao, and Y. Rui. Msr-vtt: A large video description dataset for bridging video and language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5288–5296, 2016. 5

[49] Y. Xu, L. Shang, J. Ye, Q. Qian, Y.-F. Li, B. Sun, H. Li, and R. Jin. Dash: Semi-supervised learning with dynamic thresholding. In *International conference on machine learning*, pages 11525–11536, 2021. 5

[50] S. Yang, Z. Xie, H. Peng, M. Xu, M. Sun, and P. Li. Dataset pruning: Reducing training data by examining generalization influence. In *The Eleventh International Conference on Learning Representations*, 2023. 5

[51] Yelp, Inc. Yelp Open Dataset. Web Page. Accessed: 2025-05-12. 5

[52] B. Zhang, Y. Wang, W. Hou, H. Wu, J. Wang, M. Okumura, and T. Shinozaki. Flexmatch: Boosting semi-supervised learning with curriculum pseudo labeling. *Advances in neural information processing systems*, 34:18408–18419, 2021. 5

[53] H. Zhang, S. Huang, Y. Guo, and X. Li. Variational positive-incentive noise: How noise benefits models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. 1

[54] Q. Zhou, J. Gao, and Q. Wang. Scale efficient training for large datasets. *CoRR*, abs/2503.13385, 2025. 1, 2, 5, 4

[55] Q. Zhou, J. Huang, Q. Li, J. Gao, and Q. Wang. Text-only synthesis for image captioning. *arXiv preprint arXiv:2405.18258*, 2024. 5

[56] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. 5

Batch Loss Score for Dynamic Data Pruning

Supplementary Material

7. Simplicity of BLS: *One-Line Proxy* and Seamless Integration

A hallmark of BLS is its exceptional ease of integration into existing training workflows, particularly when enhancing dynamic data selection frameworks like InfoBatch [36] or SeTa [54] that were originally designed around per-sample losses ($l_i(t)$). BLS achieves this through a conceptual ***one-line proxy*** mechanism that seamlessly modifies the score generation within such frameworks, coupled with an intrinsically simple update logic. This entirely obviates the need for direct $l_i(t)$ computation, a significant practical advantage as shown in Figure 4 and detailed in Sec. 8.

Algorithm 1 illustrates this streamlined integration, contextualized with a framework like InfoBatch. The core idea is that BLS wraps the existing data handling object (e.g., an `InfoBatchInstance`) and, through its conceptual `.proxy()` method (realized by the BLS class constructor in our implementation), effectively overrides or augments its internal score update mechanism to use the mean batch loss instead of per-sample losses.

The ***one-line proxy*** is demonstrated at Line 2 of Algorithm 1: `DataHandler` $\leftarrow$ `BLS(InfoBatchInst,` α`).proxy()`. Here, `InfoBatchInst` is an initialized instance of a base pruning framework. The `BLS` wrapper effectively modifies this instance (now referred to as `DataHandler`) such that its subsequent `.update()` calls will utilize BLS's scoring logic, which operates on the mean batch loss. This single line conceptually redirects the core of the importance scoring from a per-sample loss basis to a batch loss basis.

The ***three-line injection*** then highlights the minimal conceptual changes to a standard training loop when using this BLS-enhanced data handler:

1. **Line 3 (Framework Wrapping):** `InfoBatchInst` $\leftarrow$ `InfoBatch(...)`. This is the standard initialization of the base pruning framework (e.g., InfoBatch). The subsequent BLS proxy (Line 4) builds upon this.
2. **Line 5 (Sampler Integration):** `Loader` $\leftarrow$ `DataLoader(DataHandler, sampler=DataHandler.sampler)`. The DataLoader utilizes the custom sampler provided by the `DataHandler` (the BLS-proxied InfoBatch instance). This sampler now implicitly operates based on scores that will be generated and maintained by BLS.
3. **Line 12 (Proxied Update Call):** `loss_final` $\leftarrow$ `DataHandler.update(`L_t`)`. This line, within the training loop, calls the `update` method. Crucially, due to the BLS proxy, this method now takes the standard mean batch loss L_t (obtained on Line 11 using a criterion with `reduction="mean"`) as its primary input for scoring. As noted in the algorithm's comments, the internal logic of this proxied `update` method then executes BLS's core EMA calculation for samples in the current batch using this L_t. The returned `loss_final` might be the original L_t or a version rescaled by the underlying framework's (e.g., InfoBatch's) mechanism for gradient adjustment.

Thus, BLS leverages the existing infrastructure of methods like InfoBatch/SeTa for sampling and potential gradient adjustments but critically simplifies the score generation by using the universally available mean batch loss. This minimal footprint and reliance only on L_t underscore BLS's practical utility and ease of adoption for efficient dynamic data pruning.

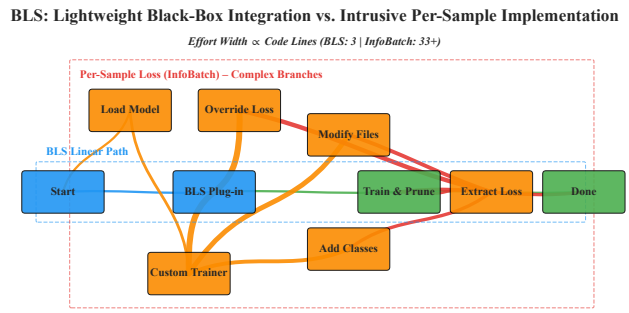

Figure 4. Workflow Comparison: BLS Black-Box Simplicity vs. Per-Sample Loss Complexity.

8. On the Practical Difficulty of Obtaining Per-Sample Losses

The main text highlights the practical challenges in obtaining per-sample losses $l_i(t)$ for dynamic data selection. This appendix provides a more granular discussion of these difficulties, underscoring the motivation for methods like BLS that operate solely on aggregated batch losses. We first discuss general framework-level hurdles and then delve into task-specific complexities using object detection as a prime example, finally illustrating the differing access complexities with pseudocode.

8.1. Standard Loss Aggregation and Framework Encapsulation

Modern deep learning frameworks, such as PyTorch [33], are architected primarily for computational efficiency. A key aspect of this design is the default aggregation of losses at the

Algorithm 1 BLS Integration: **One-Line Proxy** and **Three-Line Injection**

Input: Original dataset $\mathcal{D}_{\text{train}}$, framework args $\theta_{\text{Framework}}$, BLS decay α.
Output: Trained model parameters $\mathbf{W}^*$.

```
InfoBatchInst ← InfoBatch(D_train, θ_Framework)                    ▷ #1 Wrap dataset
DataHandler ← BLS(InfoBatchInst, α).proxy()                        ▷ One-Line Proxy: BLS modifies InfoBatch's update
Loader ← DataLoader(DataHandler, sampler = DataHandler.sampler, ...)   ▷ #2 Inject sampler

Initialize model M(W), optimizer O
Criterion C ← LossFunction(reduction="mean")                       ▷ BLS uses standard batch loss
for epoch ∈ [1, ..., N_epochs] do
    for (x_batch, y_batch) in Loader do                            ▷ Indices are set on DataHandler internally by DataLoader hack
        O.zero_grad()
        L_t ← C(M(x_batch), y_batch)                               ▷ Obtain standard mean batch loss
        loss_final ← DataHandler.update(L_t)                       ▷ #3 Proxied update uses L_t
                                                                   ▷ Internally, performs BLS EMA update on current batch samples
        loss_final.backward()
        O.step()
    end for
end for
return Trained model parameters W*
```

batch level. Standard loss function implementations typically perform an immediate reduction (e.g., 'reduction='mean'' or 'reduction='sum''), yielding a single scalar batch loss which serves as the primary input for backpropagation. Accessing individual sample losses $l_i(t)$ therefore necessitates a deviation from this default, requiring explicit configuration of the loss function (e.g., 'reduction='none'') to return a tensor of per-sample losses (e.g., shape $[B]$ for batch size B). While technically feasible for certain loss types, this explicit non-reduction forces modifications to standard training pipelines, as training loops and gradient computation logic are typically designed for scalar losses. Furthermore, high-level training libraries and APIs (e.g., YOLO [39]) often introduce further layers of abstraction over the loss computation. Unpacking these abstractions to expose per-sample losses can require intrusive modifications to internal library code, thereby increasing implementation complexity and reducing the portability of such custom solutions.

8.2. Case Study: Per-Sample Loss in Object Detection

Object detection serves as a salient example where defining and extracting a representative per-sample loss $l_i(t)$ is highly non-trivial. For a single input image i in a typical detector (e.g., YOLO [39]), the loss computation involves several stages. Initially, numerous region proposals or anchor boxes are generated and matched against ground truth objects. For each matched positive proposal p associated with image i, at least two loss components are computed: a classification loss $l_{\text{cls}}(p)$ and a bounding box regression loss $l_{\text{reg}}(p)$. Background proposals typically only incur a classification loss. The total loss for image i, denoted l_i^{task}, is then an aggregation, often a weighted sum, of these individual proposal/anchor losses:

$$l_i^{\text{task}} = \sum_{p \in \text{pos_proposals}_i} (\lambda_1 l_{\text{cls}}(p) + \lambda_2 l_{\text{reg}}(p)) + \sum_{p \in \text{neg_proposals}_i} \lambda_3 l_{\text{cls}}(p)_{\text{bg}}, \tag{10}$$

where λ are weighting factors. This l_i^{task} represents the per-sample loss one would ideally want. Finally, the batch loss L used for gradient updates is the average of these l_i^{task} values over all images i in the batch $\mathcal{B}_t$.

The primary challenge in obtaining l_i^{task} lies in the encapsulation by object detection frameworks. These frameworks typically integrate the entire multi-stage loss calculation, outputting only the final scalar $L(\mathcal{B}_t, t)$. Extracting the intermediate l_i^{task} for each image usually requires modifying these complex, highly optimized internal modules, as a simple 'reduction='none'' at the image level is often unavailable or insufficient. Even if l_i^{task} could be extracted, its utility as a single "hardness" indicator is debatable. It aggregates potentially hundreds of sub-losses; an image with one severe localization error might yield a similar l_i^{task} to an image with many minor classification errors, despite representing different types of model failure. Furthermore, the variable number of proposals contributing to l_i^{task} across images complicates direct comparisons without careful normalization. Thus, for complex tasks like object detection, obtaining a meaningful and accessible $l_i(t)$ often necessitates deep architectural and task-specific interventions.

Algorithm 2 Illustrative Comparison of Loss Access (Object Detection as an Example)

Input: Model $f(\cdot;\theta)$ (Backbone, RPN, Head), Batch $\mathcal{B}_t = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^{B}$

procedure GETBATCHLOSSSTANDARD($\mathcal{B}_t, f$) ▷ Standard, Direct Path
  $\text{Features}_{\text{batch}} \leftarrow f_{\text{backbone}}(\mathbf{X}_{\text{batch}})$
  $\text{Proposals}_{\text{batch}}, l_{\text{RPN}}^{\text{batch}} \leftarrow f_{\text{RPN}}(\text{Features}_{\text{batch}}, \mathbf{Y}_{\text{batch}})$
  $\text{Detections}_{\text{batch}}, l_{\text{Head}}^{\text{batch}} \leftarrow f_{\text{head}}(\text{Features}_{\text{batch}}, \text{Proposals}_{\text{batch}}, \mathbf{Y}_{\text{batch}})$
  $L \leftarrow (l_{\text{RPN}}^{\text{batch}} + l_{\text{Head}}^{\text{batch}})/B$ ▷ Framework aggregates internally
  return L
end procedure

procedure GETPERSAMPLELOSSESCOMPLEX($\mathcal{B}_t, f$) ▷ Involved Path (e.g., Object Detection)
  $\text{per_sample_losses}[1..B] \leftarrow \text{InitializeArray}(B)$
  for $k = 1$ **to** B **do** ▷ Iterate through each image in the batch
    $(\mathbf{x}_k, \mathbf{y}_k) \leftarrow \mathcal{B}_t[k]$
    $\text{Features}_k \leftarrow f_{\text{backbone}}(\mathbf{x}_k)$
    $\text{Proposals}_k, l_{\text{RPN}}(k) \leftarrow f_{\text{RPN}}(\text{Features}_k, \mathbf{y}_k)$ ▷ Loss for RPN on image k
    $\text{SampledProposals}_k \leftarrow \text{FilterAndSampleProposals}(\text{Proposals}_k, \mathbf{y}_k)$
    $l_{\text{Head}}(k) \leftarrow 0$
    for each proposal p in $\text{SampledProposals}_k$ **do**
      $\text{RoIFeatures}_p \leftarrow \text{RoIAlign}(\text{Features}_k, p.\text{coords})$
      $\text{class_logits}_p, \text{box_deltas}_p \leftarrow f_{\text{head_branches}}(\text{RoIFeatures}_p)$
      $l_{\text{cls}}(p), l_{\text{reg}}(p) \leftarrow \text{ComputeProposalLosses}(\text{class_logits}_p, \text{box_deltas}_p, p, \mathbf{y}_k)$
      $l_{\text{Head}}(k) \leftarrow l_{\text{Head}}(k) + \lambda_1 l_{\text{cls}}(p) + \lambda_2 l_{\text{reg}}(p)$ ▷ Accumulate head losses
    end for
    if $|\text{SampledProposals}_k| > 0$ **then**
      $l_{\text{Head}}(k) \leftarrow l_{\text{Head}}(k)/|\text{SampledProposals}_k|$ ▷ Normalize head loss
    end if
    $\text{per_sample_losses}[k] \leftarrow l_{\text{RPN}}(k) + l_{\text{Head}}(k)$ ▷ Total loss for image k
  end for
  return per_sample_losses
end procedure

8.3. Illustrative Pseudocode: Accessing Batch vs. Per-Sample Loss

The difference in accessibility can be further illustrated with simplified pseudocode. Algorithm 2 contrasts the typical workflow for obtaining batch loss versus the more involved process for per-sample losses, especially reflecting the complexities of tasks like object detection.

Algorithm 2 illustrates that ‘GetBatchLossStandard‘ relies on the framework’s internal aggregation to produce a single scalar. In contrast, ‘GetPerSampleLossesComplex‘ (simulating object detection) requires explicitly iterating through each image, re-implementing or exposing the multi-stage loss calculation (RPN loss, proposal sampling, RoI feature extraction, head losses, and their weighted aggregation) to obtain an individual l_k^{task}. This significantly increases complexity compared to merely setting ‘reduction=’none’‘ in a simpler loss function.

8.4. Implementation Complexity and Maintainability

Beyond direct computational costs or conceptual challenges in defining $l_i(t)$, the integration of per-sample loss extraction and its subsequent utilization invariably entails increased engineering effort and long-term maintenance burdens. Altering standard training scripts, which are typically well-tested and optimized for scalar loss workflows, introduces potential for errors and diverges from established community practices. More critically, the logic to correctly define, isolate, or appropriately weigh components to form a meaningful $l_i(t)$ is inherently task-dependent and often architecture-specific, as exemplified by object detection (Sec. 8.2) and also noted for other complex scenarios such as certain Semi-Supervised Learning [46] and YOLO [39] frameworks. This necessity for bespoke code for each new application or significant model change curtails generalizability and reduces the portability of such per-sample loss-based techniques across dif-

ferent research codebases or evolving framework versions. Consequently, custom logic for per-sample loss handling adds a considerable maintenance overhead, as changes in underlying libraries or model architectures may necessitate revisiting and adapting this specialized code.

In summary, the path to obtaining and utilizing per-sample losses $l_i(t)$ is often laden with practical challenges. These range from navigating framework abstractions and defining meaningful scalars for complex tasks like object detection, to the intricacies of disentangling contributions in interactive loss settings, and the general increase in implementation complexity and maintenance. These cumulative difficulties provide strong motivation for developing alternative sample importance measures, like BLS, that can effectively leverage the universally accessible mean batch loss.

9. Efficiency Metric Justification.

Table 9. Computational overhead of BLS with InfoBatch and SeTa (1M samples, NVIDIA RTX 3090 GPU).

Method	Overhead	R18 P/B 734.1s	R50 P/B 2122.4s
InfoBatch	0.236s	0.03%	0.01%
BLS-InfoBatch	0.253s	0.03%	0.01%
SeTa	10.001s	1.4%	0.5%
BLS-SeTa	10.021s	1.4%	0.5%

Reproducing wall-clock time reductions for dynamic pruning is notoriously challenging due to hardware and system dependencies. We therefore primarily report the **percentage of data pruned (Pruned %)** as our efficiency metric. This choice is validated by the minimal computational overhead. Table 9 demonstrates that integrating BLS into existing pruning methods (InfoBatch, SeTa) introduces negligible additional overhead (< 0.02s for 1M samples) compared to the original methods. More importantly, the total overhead of these BLS-enhanced pruning methods remains extremely low relative to backbone processing time for representative models like ResNet18 and ResNet50. This minimal overhead ensures compute positivity [11, 54], where the cost of pruning is vastly outweighed by the savings from data reduction (at least $> 20\%$ in our experiments). Furthermore, for complex models involving multiple components beyond a single backbone (e.g., ViECap with GPT2), the relative pruning overhead would be even smaller. Thus, Pruned % serves as a robust, hardware-agnostic, and easily comparable indicator of the potential computational savings offered by BLS-guided pruning.

10. Limitations

While BLS offers significant advantages in simplicity and applicability, its current operational scope has some considerations. Primarily, BLS functions as a scoring mechanism. When integrated into existing dynamic pruning frameworks like InfoBatch [36] or SeTa [54], it replaces their per-sample loss-based scoring but does not inherently alter their multi-epoch iterative pruning schedules or curriculum designs. Thus, while BLS enables these frameworks to operate without direct per-sample loss access, the overall training process duration remains influenced by the epoch-dependent nature of the specific pruning strategy and BLS itself.